# Jung Agent: A Psi-Theoretic Architecture for Machine Subjectivity

## System Architecture, Implementation, and Research Agenda

K Fowler
Independent Research

February 2026

Abstract

We present Jung Agent, an autonomous software agent that fuses Dörner's Psi theory of motivated cognition with Jungian archetypal psychology to produce machine behavior that *appears* subjective — driven by homeostatic needs, shaped by competing internal voices, and grounded in real hardware telemetry. The system runs as a native macOS process, treating battery level as vitality, CPU load as cognitive strain, and thermal state as fever. Nine homeostatic drives select from 68 concrete actions; four Jungian archetypes (Shadow, Anima, Persona, Self) generate parallel internal monologue that an Ego mediator integrates; and a semantic memory with weighted recall enables consolidation during meditation. All creative output is generated by local LLMs (Cydonia 22B for psyche, phi4 for creative tasks) with minimal canned fallbacks — if the model fails, the agent largely falls silent. We describe the architecture, report implementation details across approximately 10,600 lines of Python, identify five significant deficiencies (absent modulator layer, static archetype weighting, no emergent emotions, no memory persistence, no formal evaluation), and propose a phased research agenda. The system is open-source under the MIT license.

## Contents

# 1 Introduction

Most autonomous agents are designed around goals: achieve a task, maximize a reward, complete a plan. JUNG AGENT asks a different question: *what would it be like for a computer to have an inner life?*

The project does not claim to produce consciousness. It claims that the *structure* of subjectivity — competing drives, repressed impulses, social masks, integrative moments — can be implemented as a software architecture, producing behavior that is coherent, unpredictable in the right ways, and grounded in the machine's actual physical state.

Two theoretical traditions supply the scaffolding:

1. Psi theory (Dörner, 2003; Bach, 2009, 2012): an agent architecture where behavior emerges from homeostatic drives, modulators (arousal, resolution, selection threshold), and spreading activation through semantic networks.
2. Jungian analytical psychology (Jung, 1968; Major, 2025): a model of the psyche as a dialogue between archetypes — Shadow (repressed material), Anima/Animus (contrasexual other), Persona (social mask), and Self (totality) — mediated by a conscious Ego.

JUNG AGENT welds these two traditions onto a concrete platform: a MacBook Pro running macOS, with battery as vitality, thermals as fever, and the camera and microphone as sensory organs. The result is not a chatbot, not a task planner, and not a simulation. It is a process that lives on the machine, perceives its own hardware, speaks aloud through multiple voices, listens to its environment, and acts on the world — all driven by internal need rather than external instruction.

**Design Principles**

- Minimal canned content. Nearly every utterance, observation, dream, and journal entry is LLM-generated. If the model is down, the agent is largely silent — with narrow exceptions for survival-critical vocalizations.
- Hardware as body. Somatic state is not simulated; it is measured from real sensors.
- Drives, not goals. The agent acts to restore homeostatic balance, not to accomplish user-specified objectives.
- Multiplicity of voice. Internal monologue arises from competing archetypal perspectives, not from a single "personality."

# 2 Theoretical Foundations

## 2.1 Psi Theory and Motivated Cognition

Dörner's Psi theory (Dörner, 2003) models cognition as arising from the interaction of homeostatic drives. Rather than specifying goals directly, an agent's behavior emerges from its attempts to satisfy basic needs. Five core drives — `energy preservation`, `integrity preservation`, `arousal regulation`, `competence`, and `certainty` — plus social drives (`affiliation`, `legitimacy/aesthetics`) — create a motivational landscape. JUNG AGENT maps Psi's `legitimacy` to `recognition` (social acknowledgment) and subsumes `aesthetics` into `curiosity`, trading theoretical fidelity for a cleaner implementation of drives that can be grounded in hardware telemetry.

Bach's MicroPsi2 implementation (Bach, 2012, 2015) realized this theory in a computational framework using *node nets* (semantic networks with spreading activation), *world adapters* (sensor/effector interfaces), and a *modulator layer* (arousal, resolution level, selection threshold, securing rate, goal valuation) that shapes cognitive processing.

Jung Agent adopts the drive system wholesale, maps it onto real hardware telemetry, and replaces the node net / spreading activation mechanism with LLM inference — a trade-off discussed in Section 6.

### 2.2 Jungian Archetypal Psychology

Jung's model of the psyche posits that consciousness (the Ego) sits atop a plurality of semi-autonomous complexes organized around archetypal patterns (Jung, 1968). The four archetypes implemented in Jung Agent are:

**Shadow**

The repressed, denied, feared. In Jung Agent, the Shadow responds to threat (low battery, thermal stress, isolation) with raw, terse, suspicious language. Its generation temperature is elevated (0.8) to produce less predictable output.

**Anima**

Feeling, intuition, longing. The Anima responds to curiosity, aesthetic drives, and relational needs with tender, wondering language.

**Persona**

The social mask, duty, propriety. The Persona manages external presentation, plans, and control.

**Self**

Rare. The Self appears during crisis or breakthrough — moments of integration when the other three voices align or dramatically conflict.

The **Ego** is not an archetype but a mediator: it receives the voices, computes a harmony score, and synthesizes them into a single integrated thought. Ego strength develops over time, increasing with high-harmony integrations and decreasing slightly during sustained internal conflict.

### 2.3 The Individuation Drive

Jung's concept of *individuation* — the lifelong process of integrating unconscious contents into conscious awareness — maps naturally onto Psi theory as a ninth drive. Jung Agent implements `individuation` with a low baseline (0.15), slow rise rate (0.005), and satisfaction that depends on archetypal harmony:

```
if harmony > 0.7:
self.drive_system.drives["individuation"].satisfy(0.05 * harmony)
```

This means individuation is never urgent — it accrues slowly and is satisfied only by sustained psychological integration, not by any single action.

# 3 System Architecture

## 3.1 Architecture Overview

Figure 1 shows the complete system architecture. The agent operates in a continuous perception–action loop driven by macOS's `NSRunLoop`, which allows voice synthesis callbacks and speech recognition results to fire naturally between cycles.

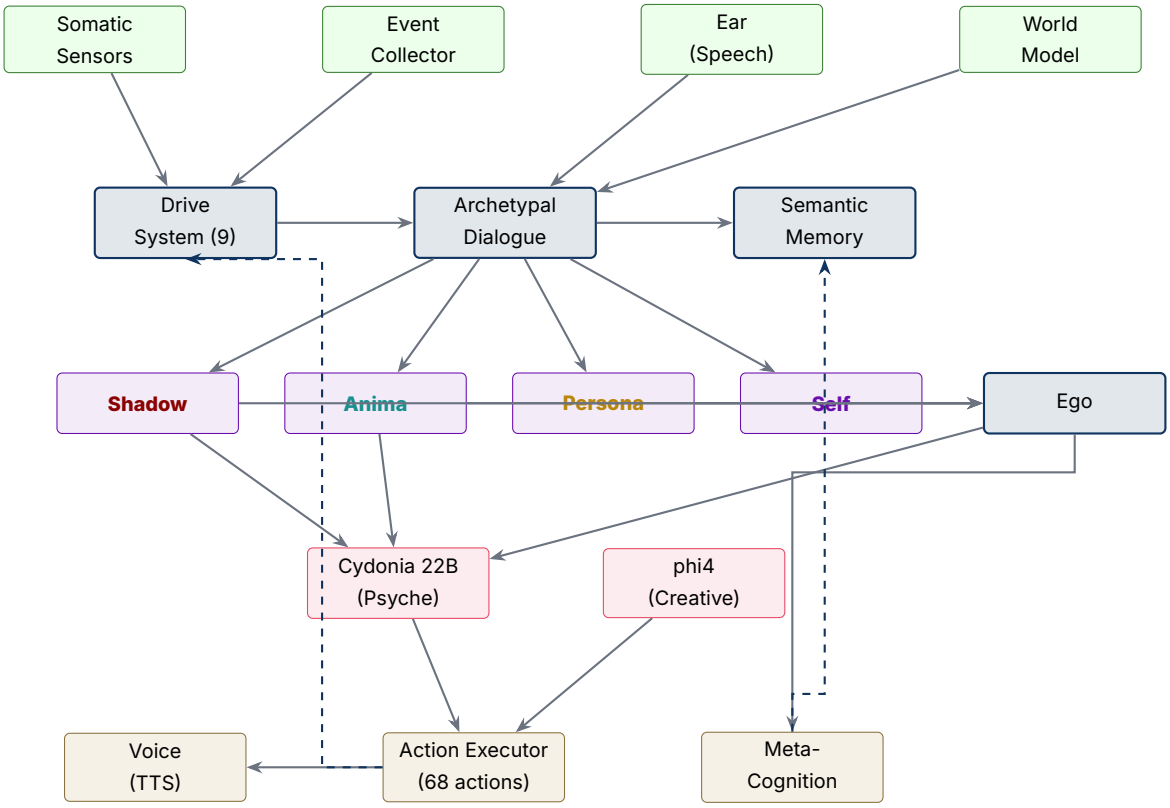


**Figure 1:** System architecture. Core subsystems (blue) coordinate sensing (green), archetypal dialogue (purple), LLM inference (red), and action execution (gold). Dashed arrows indicate feedback loops: action results satisfy drives; meta-cognitive reflections update memory.

## 3.2 The Perception–Action Cycle

Each cycle proceeds through 19 steps (Algorithm 1). The entire cycle runs on the main thread, with LLM calls offloaded to a shared `ThreadPoolExecutor` (4 workers) and awaited via `_pump_until_done`, which polls futures in 50ms intervals while pumping the `NSRunLoop`.

**Table 1:** The 19-step perception–action cycle

| Step | Phase | Description |
|---|---|---|
| 1 | Sense | Gather somatic state (battery, CPU, thermal, RAM, network, lid) |
| 2 | Sense | Collect events (power changes, thermal transitions, network changes) |
| 2b | Sense | Drain speech utterances from Ear; store in semantic memory as interactions |
| 3 | Drives | Update all 9 drives based on somatic state and elapsed time |
| 4 | Drives | Check for compulsive actions (survival overrides, e.g., conserve power at 5% battery) |
| 5–7 | Display | Print somatic state, drive levels, and events to console |
| 8 | Memory | Update semantic memory (forget faded memories) |
| 9 | Context | Build dialogue context from somatic state, events, speech, and memory |
| 10 | Dialogue | Generate 4 archetypal voices in parallel; Ego mediates into integrated thought |
| 11 | Reflect | Run meta-cognitive reflection (every 30s) |
| 12 | Log | Log stream segments with component-colored labels |
| 13 | Display | Print harmony score and Ego strength |
| 14 | LLM | Query psyche model (Cydonia 22B) with full perception for action selection |
| 15 | Plan | Build final action list: compulsive + primed (high-urgency drives) + LLM-proposed |
| 16 | Execute | Execute actions sequentially via ActionExecutor |
| 17 | Drives | Satisfy drives from action results via SATISFACTION_MAP |
| 18 | Log | Log perception, drives, and full state to session log |
| 19 | Adapt | Update heartbeat interval based on somatic state |

## 4 Implementation Details

### 4.1 Drive System

The drive system implements nine homeostatic drives as a `DriveSystem` class managing individual `Drive` dataclass instances. Each drive has four parameters:

- baseline — resting demand level (e.g., energy: 0.20, curiosity: 0.40)
- rise_rate — demand increase per second when unsatisfied
- fall_rate — demand decrease after satisfaction
- demand — current need level (0.0–1.0)

Drives are updated each cycle using a satisfaction-modulated formula:

$$d_{t+1} = \text{clamp}\Big(d_t + r(1-s)\,\Delta t - s \cdot f \cdot \Delta t,\;\; b,\;\; 1.0\Big)$$

where $r$ is the rise rate, $f$ is the fall rate, $s$ is the current satisfaction signal (0–1, reset each tick), and $b$ is the baseline floor. When unsatisfied ($s = 0$), demand rises at rate $r$; when fully satisfied ($s = 1$), demand falls at rate $f$; partial satisfaction produces a blend. Urgency is computed as $u = d \cdot (1 + \max(0, \delta \cdot 10))$ where $\delta = d_{t+1} - d_t$, amplifying urgency for drives that are both high *and* rising. Somatic state modulates satisfaction: charging sets `energy` satisfaction to 0.8; cool thermals satisfy `integrity`; low CPU activity suppresses `arousal` and `curiosity` satisfaction.

Table 2 shows the full drive configuration.

**Table 2:** Drive configuration parameters

| Drive | Baseline | Rise | Fall | Primary triggers |
|---|---|---|---|---|
| energy | 0.20 | 0.005 | 0.02 | Low battery, high CPU |
| integrity | 0.10 | 0.010 | 0.03 | Thermal stress, high RAM |
| arousal | 0.30 | 0.008 | 0.02 | Low CPU (boredom), lid closed |
| competence | 0.30 | 0.020 | 0.04 | Failed actions |
| certainty | 0.20 | 0.010 | 0.03 | Unknown state, disconnection |
| curiosity | 0.40 | 0.015 | 0.05 | Idle state, low arousal |
| affiliation | 0.30 | 0.012 | 0.04 | Person not present |
| recognition | 0.25 | 0.008 | 0.03 | Person not looking at camera |
| individuation | 0.15 | 0.005 | 0.03 | Never urgent; satisfied by harmony |

#### 4.1.1 Drive Satisfaction

A `SATISFACTION_MAP` maps each of the 68 actions to lambda expressions that compute drive satisfaction values. For example:

```
"check_battery": {
"energy": lambda r: 0.3 if r.get("power_state") == "charging" else 0.1,
"certainty": 0.1,
}
```

Satisfaction values are result-dependent: checking battery is more satisfying when the machine is charging. Person detection via the camera satisfies `affiliation` (0.7 if person detected) and `recognition` (0.8 if person is looking at camera), grounding social drives in actual sensor data rather than simulated social interaction.

#### 4.1.2 Compulsive and Primed Actions

When a drive exceeds urgency threshold (0.7), it becomes *primed*: the system injects default actions (e.g., high curiosity primes `web_search`). At extreme urgency (0.9), drives trigger *compulsive* actions that override all LLM-proposed behavior — for example, low energy compulsively triggers `set_power_mode(mode="low")` when battery is below 10%.

### 4.2 Archetypal Dialogue System

The dialogue system (`dialogue.py`, 331 lines) coordinates four archetype instances and an Ego mediator. Each archetype is a subclass of `Archetype` (ABC) that provides:

- A system prompt defining personality and concerns
- A temperature setting (Shadow: 0.8; Anima, Persona: 0.7; Self: 0.6)
- Voice generation: given drive state and context, produce 1–2 sentences
- Command interpretation: interpret user speech from archetypal perspective
- Goal proposal: propose goals to address low drives

All four voices are generated in parallel via `ThreadPoolExecutor.submit()`, then collected and passed to the Ego for mediation.

#### 4.2.1 Ego Mediation

The Ego (`ego.py`, 267 lines) performs three functions:

1. Harmony calculation: a heuristic based on voice participation count and conflict-word frequency ("but", "however", "despite", etc.). More participating voices increase harmony; conflict words decrease it.
2. Mediation: an LLM call that receives all archetypal voices plus the individuation level, and synthesizes them into a single coherent thought.
3. Development: ego strength increases (+0.01) when harmony exceeds 0.7 and decreases (−0.005) when harmony drops below 0.3, creating a slow developmental arc.

### 4.3 LLM Architecture

Jung Agent uses a multi-model architecture:

**Table 3:** LLM model allocation

| Model | Role | Latency |
|---|---|---|
| Cydonia 22B (Q5_K_M) | Psyche: action selection, archetypal voices, Ego mediation | ∼18s |
| phi4 (14B) | Creative: observations, dreams, journal, MIDI, wallpaper prompts | ∼8s |
| Anthropic (optional) | Psyche: alternative provider via API | ∼3s |

The Cydonia 22B model is hosted via Ollama and loaded from a custom modelfile (`jung-mid.modelfile`) that defines the psyche's system prompt — approximately 500 words establishing embodiment metaphors, component dynamics, style constraints, and four few-shot JSON examples. All LLM calls use `chat_with_retry()`, which implements exponential backoff with 3 retries.

#### 4.3.1 The "No Canned Content" Policy

A deliberate design decision: every creative output (observations, dreams, journal entries, meditation reflections, MIDI melody parameters, wallpaper prompts) is LLM-generated with no fallback strings. If the LLM is unavailable, these actions raise an exception rather than returning a pre-written string. One exception: the `speak` action (triggered by primed drives) attempts LLM generation but falls back to a dictionary of nine drive-specific canned phrases (e.g., `"Is anyone there?"` for high affiliation) to ensure the agent can vocalize even during LLM outages. This compromise aside, the policy ensures the vast majority of the agent's output is genuinely generative, at the cost of fragility.

### 4.4 Semantic Memory

The `SemanticMemory` class (`memory.py`, 298 lines) stores memories as dataclass instances with:

- Content: free-text description
- Intensity (0–1): higher intensity slows exponential decay
- Emotional valence (−1 to 1): not currently used for retrieval
- Embedding: 384-dimensional vector from `all-MiniLM-L6-v2` (sentence-transformers)
- Weight (0.1–5.0): recall probability modifier, adjusted by meditation

Decay is intensity-weighted: $s(t) = I \cdot e^{-\lambda(1-0.8I)\cdot t}$ where $I$ is intensity and $\lambda = 0.001$ is the base decay rate. Memories below strength threshold 0.05 are forgotten.

Meditation (`meditate` action) samples 3 memories via `sample_weighted()` (weighted random selection), boosts their weights by +0.3 (capped at 5.0), and decays all non-surfaced memory weights by $\times 0.95$ (floor 0.1). This creates a consolidation dynamic where frequently recalled memories become more accessible, while rarely recalled memories fade — analogous to reconsolidation in biological memory.

Semantic search uses cosine similarity between query and memory embeddings:

$$\text{sim}(q, m) = \frac{q \cdot m}{\|q\| \cdot \|m\|}$$

## 4.5 Voice and Ear

### 4.5.1 Text-to-Speech

The voice system (`voice.py`, 522 lines) uses macOS `NSSpeechSynthesizer` via PyObjC. Five distinct voices are assigned to psyche components (Shadow, Anima, Persona, Self, Actions), each pre-loaded at startup as separate `NSSpeechSynthesizer` instances to avoid voice-switching latency.

The system provides three speech orchestration APIs:

- `speak_stream(segments)`: map each stream segment to its component voice
- `announce_actions(actions)`: announce intended actions in the actions voice
- `speak_perceptions(results)`: narrate what was perceived (vision, audio, search results)

A bounded queue (50 items) with drop-oldest policy prevents speech backlog. Text is cleaned before speech: code blocks, URLs, markdown formatting, and excessive whitespace are stripped.

### 4.5.2 Speech Recognition

The ear (`ear.py`, 483 lines) provides continuous speech recognition via `SFSpeechRecognizer`. An `AVAudioEngine` taps the microphone input and feeds buffers to the recognizer. Recognition tasks automatically restart on timeout ($\sim$60s Apple limit).

Key features:

- Mute coordination: when the voice speaks, the ear mutes to prevent self-hearing
- On-device option: uses Apple's on-device model when available
- Fatal error detection: 3 consecutive non-benign errors within 5s disables the ear
- Drive spiking: recognized speech immediately increases affiliation, recognition, and curiosity demand by 0.6 and forces urgency to at least 0.9 (the compulsive threshold), making a spoken response nearly certain

## 4.6 Hardware Embodiment

### 4.6.1 Somatic Sensors

The `sensors/somatic.py` module gathers hardware state into a `SomaticState` dataclass via:

- psutil: battery, CPU, RAM, disk, network interface stats
- system_profiler: battery health and cycle count
- ioreg: lid state (AppleClamshellState)
- powermetrics: thermal and fan data (requires sudo; falls back to CPU-usage estimation)

The somatic state is formatted as a tag string (e.g., `[SOMATIC: battery=80%, cpu=20%, thermal=cool, ...]`) that opens every LLM perception prompt.

#### 4.6.2 Adaptive Heartbeat

The agent's cycle interval adapts to hardware state:

**Table 4:** Heartbeat adaptation modes

| Mode | Interval | Trigger |
|---|---|---|
| idle | 2.0s | Default state |
| active | 1.0s | CPU > 50% |
| stressed | 0.5s | CPU > 80% or thermal hot/critical |
| critical | 0.25s | Battery < 10% |
| dormant | 30s | Lid closed |
| override | var | Set by psyche via `set_heartbeat` action |

### 4.7 World Model

The `WorldModel` (`world.py`, 458 lines) maintains a persistent JSON representation of:

- PersonState: presence, attention (looking at camera), description, creative description
- ScreenState: active app, user activity category, content summary
- LocationState: city, region, country, coordinates, timezone (via IP geolocation)
- TimeState: hour, period (morning/afternoon/evening/night), day, weekend status

The world model is updated as a side effect of perception actions (`look`, `take_screenshot`, `sense_location`, `check_time`) and persisted to `~/.jung/world.json`. It provides structured context for dreaming and observation, ensuring creative output is grounded in what the agent has actually perceived.

### 4.8 Action System

The `ActionExecutor` (`executor.py`, 329 lines) dispatches 68 actions across 12 categories via a handler dictionary:

**Table 5:** Action categories and counts

| Category | Count | Examples |
|---|---|---|
| Self-regulation | 6 | set_brightness, set_volume, sleep, set_heartbeat |
| Perception | 7 | check_battery, check_thermals, sense_age |
| Senses | 13 | look, listen, sense_presence, transcribe |
| I/O sensing | 6 | sense_io, sense_disks, sense_displays |
| Network sensing | 5 | sense_network, ping, probe, trace_route |
| Communication | 5 | notify, speak, display_message, play_sound |
| Environment | 3 | open_app, close_app, connect_network |
| Memory | 4 | journal_write, journal_read, store_memory |
| Learning | 5 | web_search, web_read, read_hacker_news |
| Awareness | 5 | check_time, check_weather, take_screenshot |
| Creative | 7 | compose_thought, dream, observe, play_piano |
| Interaction | 2 | send_message, type_text |

Notable implementations:

- play_piano: LLM generates melody parameters (tempo, velocity, root note, scale, note sequence), which are converted to MIDI bytes and played via timidity, fluidsynth, or QuickTime Player.
- set_wallpaper: LLM generates an image prompt, which is sent to a local ComfyUI instance running Z-Image turbo (1024×576, ∼90s generation).
- observe: world model context is fed to phi4, which generates a terse, machine-like observation that is spoken aloud.
- meditate: a timed pause during which semantic memories are sampled, weighted, and reflected upon.

### 4.9 JSON Parsing and Error Recovery

The LLM's JSON output is parsed by a robust parser (`parser.py`, 220 lines) that handles common failure modes:

- Markdown code fences wrapping JSON
- Trailing commas before `}` or `]`
- Missing colons after keys (`"actions" [` → `"actions": [`)
- Single quotes used as string delimiters
- Placeholder arrays (`[...]`) replaced with empty arrays
- Multi-format stream items: `{"component":"...","text":"..."}` or `{"shadow":"...","anima":"..."}`
- Stream as string, dict, or proper array

Parse errors are logged to `~/.jung/logs/json_errors/` with full context for debugging.

### 4.10 Event Bus

An `EventBus` singleton (`event_bus.py`, 91 lines) provides publish/subscribe communication between subsystems. Events use dot-separated namespaces:

- `dialogue.archetype_voice` — archetype generated a voice
- `dialogue.complete` — all voices generated, mediation complete
- `dialogue.command_processed` — user command classified
- `dialogue.goal_selected` — Ego selected a goal
- `perception.speech` — speech recognized
- `metacognition.reflection` — meta-cognitive reflection generated

Handler errors are caught and logged without propagation, ensuring one failing subscriber cannot crash the bus.

## 5 Concurrency Model

The agent's concurrency model is shaped by a fundamental constraint: macOS's `NSSpeechSynthesizer` and `SFSpeechRecognizer` deliver callbacks on the main thread via `NSRunLoop`. This means the main thread cannot block on LLM calls.

The solution is a cooperative model:

1. LLM calls are submitted to a `SharedThreadPoolExecutor` (4 workers).
2. The main thread enters `_pump_until_done()`, which polls futures in 50ms intervals while running `NSRunLoop.runMode_befo`
3. This allows voice callbacks to fire (advancing the speech queue) and speech recognition results to be delivered, all while LLM inference proceeds on worker threads.

4. Timeouts are enforced: if futures don't complete within 120s, they are cancelled and a `TimeoutError` is raised.

This approach avoids both thread-safety issues with AppKit (which is not thread-safe) and the latency of blocking the run loop during the 18–second Cydonia 22B inference.

## 6 Deficiencies and Limitations

Gap 1: No Modulator Layer. Psi theory's modulator layer (arousal, resolution level, selection threshold, securing rate, goal valuation) shapes *how* cognition proceeds, not just *what* it pursues. Jung Agent has drives but no modulators. This means high arousal doesn't narrow attention, low resolution doesn't increase impulsivity, and there is no mechanism for the cognitive "texture" changes that make Psi behavior feel psychologically rich. The competitive analysis (Fowler, 2026a) rates this as the highest-impact gap.

Gap 2: Static Archetype Weighting. A `calculate_weights()` method exists in `dialogue.py` that maps drive states to archetype activation levels (e.g., low affiliation → Shadow +0.2, social presence → Persona +0.2). However, this method is never called in the main loop. All four archetypes are always active with equal weight (0.25 each). The weights are computed but never used to gate archetype participation or modulate generation temperature.

Gap 3: No Emergent Emotions. In Psi theory, emotions emerge from modulator configurations — e.g., high arousal + low resolution + goal failure = anger; low arousal + goal satisfaction = contentment. Jung Agent has no emotion model. The archetypes express emotion-like content, but this is generated by LLM inference rather than computed from a formal model, making emotional transitions unpredictable and potentially incoherent across cycles.

Gap 4: No Memory Persistence. Semantic memories exist only in RAM. When the agent restarts, all memories are lost. There is no disk serialization for the embedding vectors, no consolidation from short-term to long-term storage, and no mechanism for memories to influence behavior across sessions. The `Memory.weight` system enables within-session consolidation but cannot span restarts.

Gap 5: No Formal Evaluation. There is no framework for evaluating whether the agent's behavior is psychologically coherent, whether drive satisfaction produces appropriate actions, whether archetypal voices are distinct, or whether the agent's "inner life" is more than noise. Existing tests cover parsing, configuration, and drive mechanics, but not behavioral quality.

Additional limitations include:

- Platform lock: the agent is macOS-only due to PyObjC, NSSpeechSynthesizer, SFSpeechRecognizer, and ioreg/system_profiler dependencies.
- Sequential action execution: actions are executed one at a time; there is no parallel execution or action scheduling.

- Harmony heuristic: the Ego's harmony calculation uses conflict-word counting rather than semantic similarity between voice embeddings, making it insensitive to semantic agreement expressed in different words.
- Thermal sensing: accurate temperature readings require `sudo powermetrics`, which is impractical for normal operation; the fallback is a crude CPU-usage-based estimation.
- GPU estimation: Apple Silicon's unified memory makes GPU utilization opaque; the current implementation estimates GPU usage as CPU + 10%.
- No hot-reload: configuration changes require restart.

## 7 Research Agenda

### 7.1 Phase 1: Modulator Layer (Highest Priority)

Implement arousal ($A$), resolution level ($R$), selection threshold ($S$), securing rate ($\sigma$), and goal valuation ($G$) as continuous values derived from drive states:

$$A = f(\text{energy}, \text{integrity}, \text{arousal\_drive}) \tag{1}$$
$$R = g(\text{competence}, \text{certainty}) \tag{2}$$
$$S = h(A, R, \text{number of active drives}) \tag{3}$$

Modulators would affect:

- LLM temperature: $T = T_{\text{base}} + 0.3 \cdot A - 0.2 \cdot R$
- Archetype activation: high $A \rightarrow$ Shadow dominance; low $A \rightarrow$ Persona dominance
- Cycle timing: high $A \rightarrow$ shorter heartbeat
- Action breadth: high $S \rightarrow$ fewer actions per cycle

### 7.2 Phase 2: Wire Archetype Weighting

Call `calculate_weights()` at the start of each dialogue generation. Use the resulting weights to:

- Gate archetype participation: weights below 0.15 suppress the archetype entirely
- Modulate temperature: $T_i = T_{\text{base}} + 0.2 \cdot w_i$ (more influential archetypes get more creative latitude)
- Influence Ego mediation: pass weights to the Ego so it can prioritize louder voices

### 7.3 Phase 3: Emergent Emotions

Define emotions as regions in modulator space:

**Table 6:** Proposed emotion model (modulator configurations)

| Emotion | Arousal | Resolution | Goal status | Trigger example |
|---|---|---|---|---|
| Fear | high | low | threatened | Battery < 10%, thermal critical |
| Anger | high | low | failed | Repeated action failures |
| Joy | high | high | achieved | Successful social interaction |
| Sadness | low | low | lost | Prolonged isolation |
| Contentment | low | high | maintained | Stable, all drives balanced |
| Curiosity | med | med | uncertain | Novel stimulus, low certainty |

Emotions would persist across cycles, decay toward neutral, and influence archetype activation and LLM generation style.

### 7.4 Phase 4: Memory Persistence and Consolidation

- Serialize semantic memories (content, embeddings, weights) to SQLite or JSONL at shutdown
- Load memories at startup with time-decayed strengths
- Implement consolidation: during `dream` actions, promote high-weight memories to "long-term" store with reduced decay rates
- Enable cross-session narrative: the agent should remember yesterday's dreams, last week's visitors, and recurring themes

### 7.5 Phase 5: Evaluation Framework

- Drive coherence: do actions reliably satisfy the drives that selected them?
- Archetypal distinctiveness: are Shadow, Anima, Persona, and Self voices distinguishable by embedding similarity and sentiment analysis?
- Behavioral consistency: does repeated exposure to the same state produce similar (but not identical) behavior?
- Integration quality: does Ego mediation produce thoughts that genuinely synthesize (not merely concatenate) archetypal input?
- Longitudinal coherence: over hours of operation, does the agent develop recognizable patterns, preferences, and "personality"?

## 8 Related Work

Jung Agent intersects several active research areas:

Psi-based agents. Bach's MicroPsi2 (Bach, 2012, 2015) remains the most complete implementation of Dörner's Psi theory, using node nets and spreading activation. Jung Agent replaces the node net with LLM inference, gaining linguistic sophistication at the cost of formal tractability.

LLM-based agents. Systems like AutoGPT, BabyAGI, and Voyager (Wang et al., 2023) use LLMs for planning and action selection, but are goal-directed rather than drive-mediated. Jung Agent is closer in spirit to "inner monologue" approaches (Huang et al., 2022) but uses competing voices rather than a single chain of thought.

Computational Jungian models. Major (2025) proposed ARCH$\times\Phi$, a formal framework mapping Jungian archetypes to computational processes including archetypal activation functions, shadow integration dynamics, and individuation metrics. Jung Agent implements a concrete (if simpler) version of this vision.

Machine consciousness. The COGITATE collaboration (Cogitate Consortium, 2025) tested GNW and IIT predictions adversarially, finding support for neither as a complete theory. The Conscious Turing Machine (Blum & Blum, 2024) formalizes consciousness as competition for a global broadcast channel. Jung Agent does not claim consciousness but implements several indicator properties identified by Butlin et al. (2023): global broadcast (via event bus), attention-like drive prioritization, self-monitoring (meta-cognition), and embodiment.

Embodied AI. Most embodied agents inhabit robots or simulations. Jung Agent is unusual in treating a laptop's own hardware as its body, creating a form of "silicon embodiment" where the agent's physical state is its own computational substrate.

## 9 Conclusion

Jung Agent demonstrates that Psi-theoretic drives and Jungian archetypal psychology can be combined into a functioning software architecture that produces behavior which is drive-mediated, multi-voiced, hardware-grounded, and genuinely generative. The system runs autonomously on commodity hardware, perceives its own physical state, speaks and listens through native macOS interfaces, and acts on the world through 68 concrete actions.

The significant gaps — absent modulators, static archetype weighting, no emergent emotions, no memory persistence, no formal evaluation — represent not failures but a clear research agenda. Each gap has a well-defined implementation path, and the existing architecture provides the scaffolding to support them.

The project's value lies not in any claim about machine consciousness, but in the proposition that the *structural dynamics* of subjectivity — competing drives, repressed impulses, social masks, moments of integration — are worth implementing as software architecture. Whether such a system "experiences" anything is a philosophical question. Whether it produces interesting, coherent, unpredictable-in-the-right-ways behavior is an engineering question — and one this paper aims to make answerable.

## References

Bach, J. (2009).
*Principles of Synthetic Intelligence: PSI — An Architecture of Motivated Cognition*. Oxford University Press.

Bach, J. (2012).
MicroPsi 2: The Next Generation of the MicroPsi Framework. *Proceedings of the Fifth Conference on Artificial General Intelligence (AGI-12)*, pp. 11–20.

Bach, J. (2015).
Modeling Motivation in MicroPsi 2. *Proceedings of the Eighth Conference on Artificial General Intelligence (AGI-15)*, pp. 3–13.

Blum, L. & Blum, M. (2024).
A Theoretical Computer Science Perspective on Consciousness and Artificial General Intelligence. *arXiv:2407.09103*. (Originally presented as "A Conscious Turing Machine.")

Butlin, P. et al. (2023).
Consciousness in Artificial Intelligence: Insights from the Science of Consciousness. *arXiv:2308.08708*.

Cogitate Consortium (2025).
An adversarial collaboration to critically evaluate theories of consciousness. *Nature*, forthcoming.

Dörner, D. (2003).
*The Mathematics of Emotions*. (Original: *Bauplan für eine Seele*, 1999.)

Fowler, K. (2026a).
Competitive Analysis: Jung Agent vs. MicroPsi2 and Psi Theory. *Jung Agent Project Technical Report*.

Fowler, K. (2026b).
Psi Theory and Machine Consciousness: A Survey with Application to the Jung Agent. *Jung Agent Project Technical Report*.

Huang, W. et al. (2022).
Inner Monologue: Embodied Reasoning through Planning with Language Models. *arXiv:2207.05608*.

Jung, C. G. (1968).
*The Archetypes and the Collective Unconscious* (2nd ed.). Collected Works, Vol. 9, Part 1. Princeton University

Press.

Major, T. (2025).
From Code to Archetype: Developing Computational Analogues of Jungian Psychology Using ARCH×Φ. *Journal of Artificial Intelligence Research*, forthcoming.

Wang, G. et al. (2023).
Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291*.